SAÉ S3.A.01 - Développement d'une Application

Sprint 2 - Première génération de fractale

Romain Wallon

IUT de Lens - BUT 2 Informatique

2026-2027

UNIVERSITÉ D'ARTOIS

IUT Institut Universitaire Technologie LENS

Sprint 1 - Rétrospective

Pour les validations techniques, vérifiez bien ce qui est demandé dans le *sprint*, et prévenez-nous dès que vous avez terminé ces parties (pas besoin d'attendre que tout le reste soit prêt)

Sprint 1 - Rétrospective

Pour les validations techniques, vérifiez bien ce qui est demandé dans le *sprint*, et prévenez-nous dès que vous avez terminé ces parties (pas besoin d'attendre que tout le reste soit prêt)

Veillez à bien respecter les consignes pour la gestion de projet : création d'un jalon pour le *sprint*, lien entre les tickets et le jalon, mention des tickets dans les messages de *commit*, *etc.*

Sprint 1 - Rétrospective

Pour les validations techniques, vérifiez bien ce qui est demandé dans le *sprint*, et prévenez-nous dès que vous avez terminé ces parties (pas besoin d'attendre que tout le reste soit prêt)

Veillez à bien respecter les consignes pour la gestion de projet : création d'un jalon pour le *sprint*, lien entre les tickets et le jalon, mention des tickets dans les messages de *commit*, *etc.*

Et vous, avez-vous des remarques sur le sprint précédent ?

But du *sprint*

À l'issue de ce *sprint*, vous devez pouvoir générer en niveaux de gris une image représentant la fractale de Julia pour la constante $c = -0.4 + 0.6i$

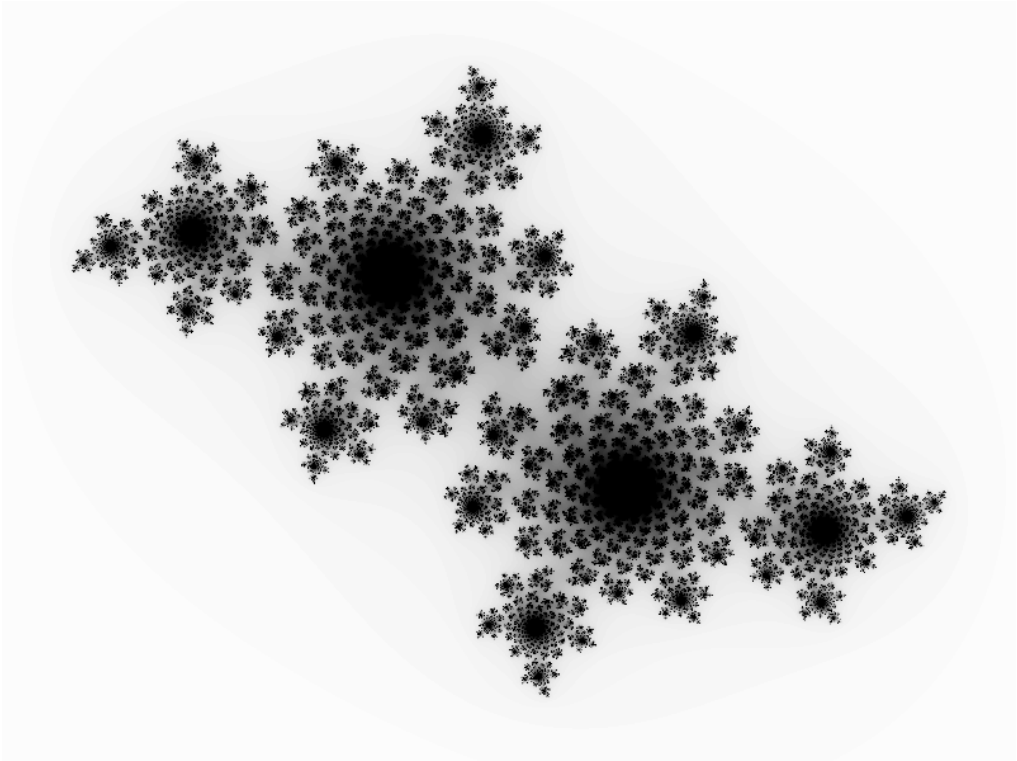


Figure – Fractale de Julia pour $c = -0.4 + 0.6i$

« Zoom » sur le plan complexe

Pour représenter la fractale de Julia, il est nécessaire de pouvoir « zoomer » sur le plan complexe, car la fractale s'étend sur une petite portion de ce plan.

« Zoom » sur le plan complexe

Pour représenter la fractale de Julia, il est nécessaire de pouvoir « zoomer » sur le plan complexe, car la fractale s'étend sur une petite portion de ce plan.

Vous allez donc devoir définir un décorateur de `IComplexPlane` qui permettra de modifier l'échelle du plan complexe, en multipliant le nombre complexe initialement fourni par la méthode `asComplex()` par un facteur de zoom z

« Zoom » sur le plan complexe

Pour représenter la fractale de Julia, il est nécessaire de pouvoir « zoomer » sur le plan complexe, car la fractale s'étend sur une petite portion de ce plan.

Vous allez donc devoir définir un décorateur de `IComplexPlane` qui permettra de modifier l'échelle du plan complexe, en multipliant le nombre complexe initialement fourni par la méthode `asComplex()` par un facteur de zoom z

Il faudra multiplier par une valeur comprise entre 0 *et* 1 *pour agrandir l'image (dans la démonstration, vous utiliserez un zoom de* 0.004*)*

Stratégie de sélection de couleur

Pour déterminer la couleur à utiliser pour un pixel de l'image, il faut itérer sur la suite commençant en ce point, et compter le nombre d'itérations effectuées avant que la suite ne diverge (ou que le nombre maximum d'itérations soit atteint)

Stratégie de sélection de couleur

Pour déterminer la couleur à utiliser pour un pixel de l'image, il faut itérer sur la suite commençant en ce point, et compter le nombre d'itérations effectuées avant que la suite ne diverge (ou que le nombre maximum d'itérations soit atteint)

Le nombre d'itérations réalisées déterminer la couleur

Stratégie de sélection de couleur

Pour déterminer la couleur à utiliser pour un pixel de l'image, il faut itérer sur la suite commençant en ce point, et compter le nombre d'itérations effectuées avant que la suite ne diverge (ou que le nombre maximum d'itérations soit atteint)

Le nombre d'itérations réalisées déterminer la couleur

Pour varier les couleurs, il vous faut définir une stratégie dont le but sera de retourner une couleur à partir d'un nombre d'itérations

Stratégie de sélection de couleur

Pour déterminer la couleur à utiliser pour un pixel de l'image, il faut itérer sur la suite commençant en ce point, et compter le nombre d'itérations effectuées avant que la suite ne diverge (ou que le nombre maximum d'itérations soit atteint)

Le nombre d'itérations réalisées déterminer la couleur

Pour varier les couleurs, il vous faut définir une stratégie dont le but sera de retourner une couleur à partir d'un nombre d'itérations

Pour obtenir un niveau de gris, vous pouvez créer une couleur dont les composantes rouge, verte et bleue sont égales au ratio du nombre d'itérations effectuées par rapport au nombre maximum d'itérations

Créer une image

Pour créer une image, vous devez définir un adaptateur qui implémentera l'interface `IFractalImage` à partir d'une instance de `BufferedImage`

Créer une image

Pour créer une image, vous devez définir un adaptateur qui implémentera l'interface `IFractalImage` à partir d'une instance de `BufferedImage`

La documentation de cette classe et le sujet vous expliquent comment faire le lien entre les méthodes de l'interface et celles de la classe `BufferedImage`

Créer une image

Pour créer une image, vous devez définir un adaptateur qui implémentera l'interface `IFractalImage` à partir d'une instance de `BufferedImage`

La documentation de cette classe et le sujet vous expliquent comment faire le lien entre les méthodes de l'interface et celles de la classe `BufferedImage`

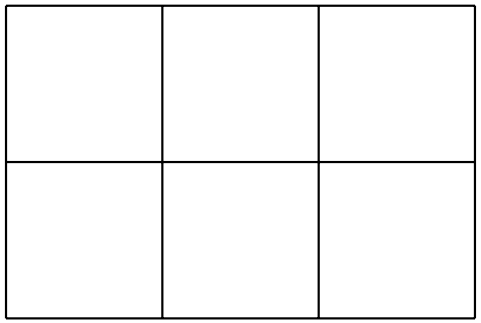
Attention ! L'ordre des coordonnées dans `BufferedImage` est différent de celui des utilisés dans `IFractalImage`

Génération d'une fractale

Pour générer une fractale, il faut évaluer la suite considérée en chaque point de l'image, et déterminer la couleur à utiliser en comptant le nombre d'itérations réalisées

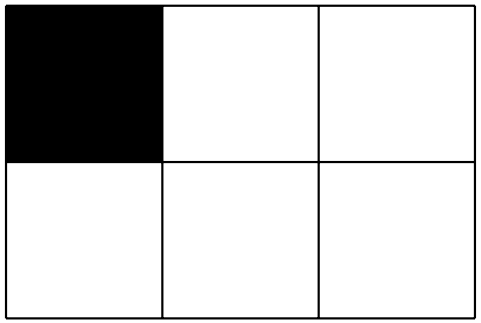
Génération d'une fractale

Pour générer une fractale, il faut évaluer la suite considérée en chaque point de l'image, et déterminer la couleur à utiliser en comptant le nombre d'itérations réalisées



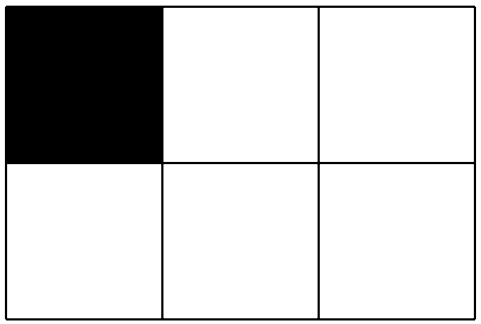
Génération d'une fractale

Pour générer une fractale, il faut évaluer la suite considérée en chaque point de l'image, et déterminer la couleur à utiliser en comptant le nombre d'itérations réalisées



Génération d'une fractale

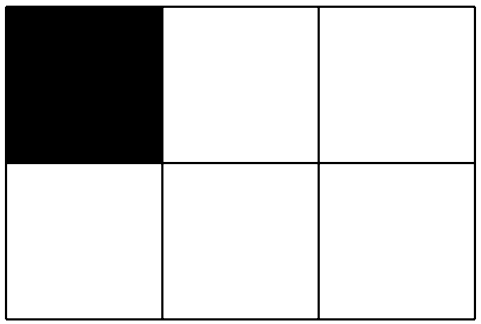
Pour générer une fractale, il faut évaluer la suite considérée en chaque point de l'image, et déterminer la couleur à utiliser en comptant le nombre d'itérations réalisées



1. $(row, col) \rightarrow z_0 = a + ib$

Génération d'une fractale

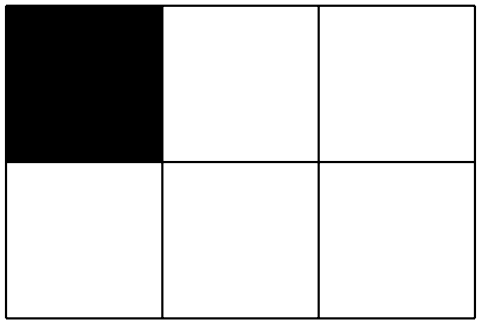
Pour générer une fractale, il faut évaluer la suite considérée en chaque point de l'image, et déterminer la couleur à utiliser en comptant le nombre d'itérations réalisées



1. $(row, col) \rightarrow z_0 = a + ib$
2. parcours de la suite : $z_0, z_1, z_2, \ldots, z_n$

Génération d'une fractale

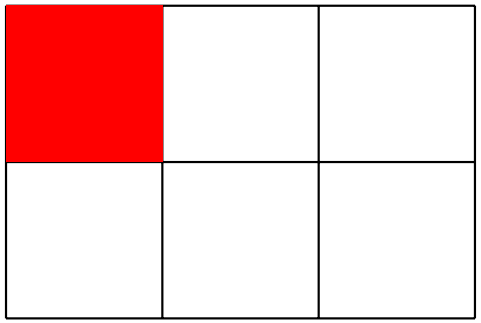
Pour générer une fractale, il faut évaluer la suite considérée en chaque point de l'image, et déterminer la couleur à utiliser en comptant le nombre d'itérations réalisées



1. $(row, col) \rightarrow z_0 = a + ib$
2. parcours de la suite : $z_0, z_1, z_2, \ldots, z_n$
3. n itérations : couleur ?

Génération d'une fractale

Pour générer une fractale, il faut évaluer la suite considérée en chaque point de l'image, et déterminer la couleur à utiliser en comptant le nombre d'itérations réalisées



1. $(row, col) \rightarrow z_0 = a + ib$
2. parcours de la suite : $z_0, z_1, z_2, \ldots, z_n$
3. n itérations : couleur ?

Initialiser la suite

Au moment d'initialiser le générateur, vous ne pouvez pas encore instancier la suite, car vous ne connaissez pas encore le nombre complexe initial z_0 (il faudra initialiser une nouvelle suite pour chaque pixel de l'image)

Initialiser la suite

Au moment d'initialiser le générateur, vous ne pouvez pas encore instancier la suite, car vous ne connaissez pas encore le nombre complexe initial z_0 (il faudra initialiser une nouvelle suite pour chaque pixel de l'image)

Pour pouvoir différer l'instanciation, il vous faudra définir une expression lambda qui doit créer une nouvelle instance de la liste à partir d'un z_0 donné et du nombre de termes à considérer :

```
(z0, nbTerms) -> new JuliaSequence(z0, c, nbTerms)
```

Initialiser la suite

Au moment d'initialiser le générateur, vous ne pouvez pas encore instancier la suite, car vous ne connaissez pas encore le nombre complexe initial z_0 (il faudra initialiser une nouvelle suite pour chaque pixel de l'image)

Pour pouvoir différer l'instanciation, il vous faudra définir une expression lambda qui doit créer une nouvelle instance de la liste à partir d'un z_0 donné et du nombre de termes à considérer :

```
(z0, nbTerms) -> new JuliaSequence(z0, c, nbTerms)
```

Le type de cette expression lambda est `BiFunction<IComplex, Integer, IComplexSequence>`, qui définit une méthode `apply(IComplex, Integer)` qui créera l'instance de la suite à utiliser

Attendus de ce *sprint*

- Implémentation du décorateur permettant de « zoomer » sur le plan complexe
- Définition d'une stratégie de sélection de couleur à partir d'un nombre d'itérations
- Implémentation de cette stratégie pour définir une palette en niveaux de gris
- Définition de l'adaptateur permettant de manipuler une instance de `BufferedImage` en utilisant les méthodes de l'interface `IFractalImage`
- Implémentation de l'algorithme de génération d'une fractale
- Définition de la méthode `main` permettant de générer une image de fractale et de la sauvegarder dans un fichier
- Tests unitaires associés à ces fonctionnalités

Validations techniques à réaliser en cours de *sprint*

- Implémentation du décorateur de « zoom »
- Implémentation de la palette de couleurs en niveaux de gris
- Implémentation de l'adapteur d'`IFractalImage` pour `BufferedImage`
- Implémentation de l'algorithme de génération

Validations techniques à réaliser en cours de *sprint*

- Implémentation du décorateur de « zoom »
- Implémentation de la palette de couleurs en niveaux de gris
- Implémentation de l'adapteur d'`IFractalImage` pour `BufferedImage`
- Implémentation de l'algorithme de génération

N'attendez pas la dernière minute pour faire valider ces points techniques : prévenez-nous dès que vous êtes prêts afin que nous puissions faire les validations de tout le monde dans les temps

Démonstration finale

À l'issue de ce *sprint*, vous devez pouvoir générer une image de taille de 600×800 pixels représentant la fractale de Julia avec $c = -0.4 + 0.6i$ en niveau de gris, avec un nombre maximum d'itérations de 100 et un zoom de 0.004

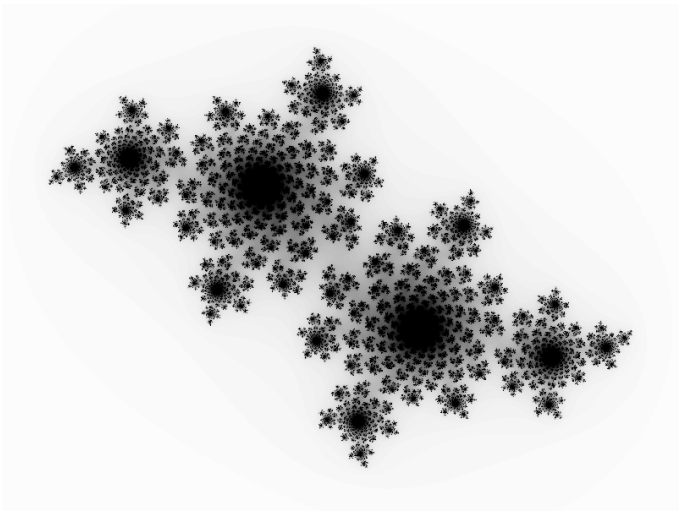


Figure – Fractale de Julia pour $c = -0.4 + 0.6i$

SAÉ S3.A.01 - Développement d'une Application

Sprint 2 - Première génération de fractale

Romain Wallon

IUT de Lens - BUT 2 Informatique

2026-2027

UNIVERSITÉ D'ARTOIS

IUT Institut Universitaire Technologie LENS